

Fig. 2: Validation domain and characteristics

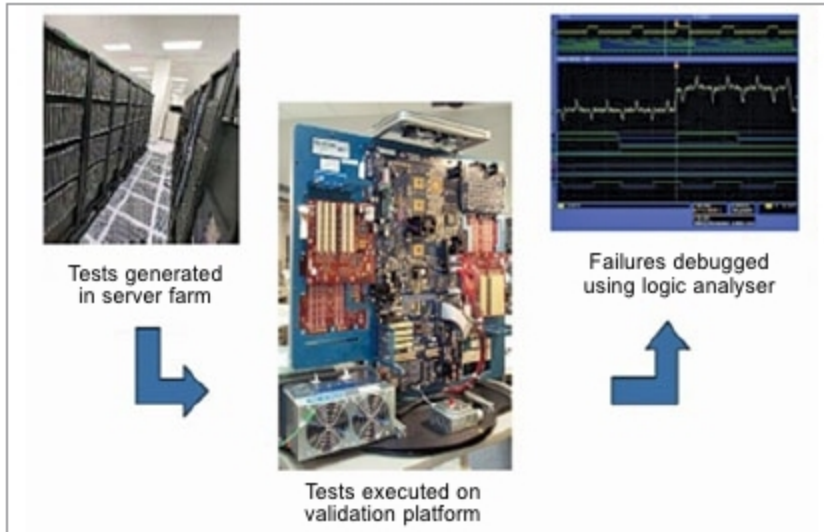


Fig. 3: Platform validation infrastructure

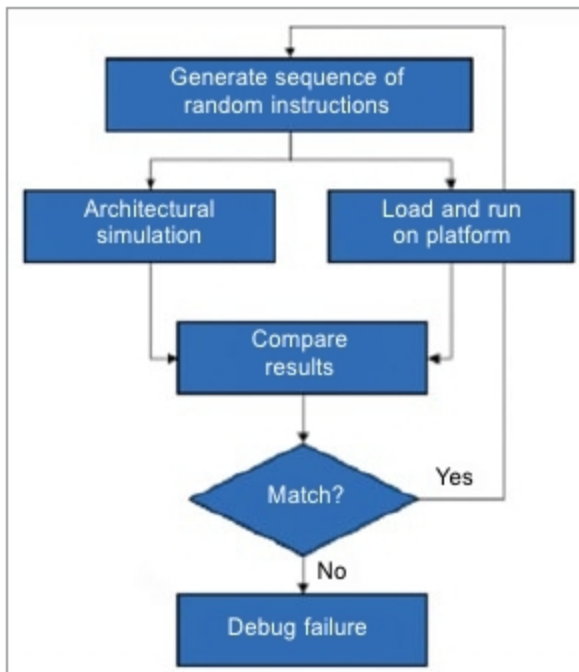


Fig. 4: Random instruction testing

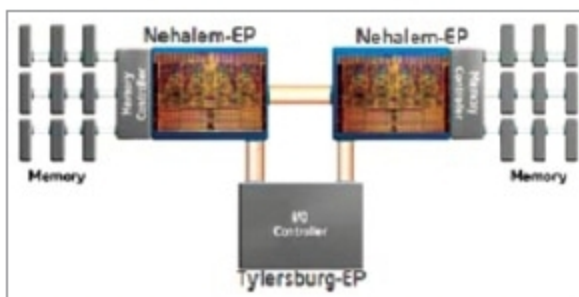


Fig. 5: Memory sub-system validation

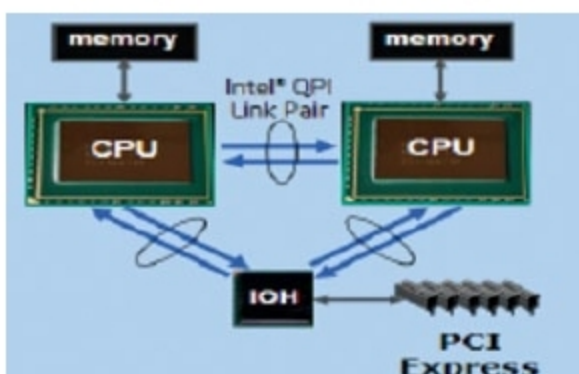


Fig. 6: I/O concurrency

Post-simulation (platform) strengths show the actual target platform—two per cent of logic bugs found, ten per cent of circuit bugs found. Its limits are difficult debugging and expensive bug fix.

Focus areas of post-simulation are complementary to pre-simulation—exploiting post-simulation benefits (many cycles, platform-level interactions), ISA and features, memory sub-system/hierarchy, platform power state transitions, I/O concurrency, I/O margin characterisation and core circuit bug hunting.

Functional bug hunting

1. ISA architecture/micro-architecture testing is based on biased random schemes, such as random generation of instructions, checking based on architectural simulation, CPU core intensive, high throughput of random tests but typically low I/O stress and random power state transition injection.

2. Feature-oriented directed/random tests including paging, TLB and virtualisation.

Random instruction testing

1. Wide coverage of CPU/GPU cores

2. Strengths:

- Finds subtle uarch bugs
- Stresses CPU pipeline boundary conditions

• Good at finding micro-code bugs

- High throughput core testing

3. Limits:

- Low I/O stress
- Need to complement with memory sub-systems tests
- Requires servers to generate instruction seeds

Memory sub-system validation

Options include:

1. Random and directed/random memory test strategy

2. Memory channel intensive

3. Based on multi-core and multi-

processor configurations

4. When the target is standard and symmetric having multi-processor attributes like cache coherency, consistency and synchronisation, and memory ordering

I/O concurrency

1. The strategy here is to simultaneously load all platform buses.

- Quick path interconnect
- DDR3 memory channels
- PCI Express Gen2
- USB, SSD
- SATA, PATA
- Display

2. Use of directed/random and biased random test generators

3. Test cards used to provide determinism

Compatibility and standards

1. Industry-standard operating systems (OSes), applications and peripherals are used to verify:

- Platform and component behavioural correctness
- Legacy compatibility of OS, applications and peripherals

2. Test configuration models and end-users

- Mobile and desktop client systems
- Blade and enterprise-level server systems

• Fully-integrated platforms: CPU, chipsets, BIOS, OSes and applications

3. Highly-stressful configurations, hunting for both functional and performance bugs

How circuit bugs appear

Some of the ways bugs appear:

1. Circuit bugs appear as DPMs—not all die or behave the same way

2. Taxonomy (classification)

(a) Timing convergence bugs

- Circuit operates too slow (speed-path),

- Circuit operates too fast (minimum delay), or

- Circuit fails due to timing of multiple converging signals (race)

(b) Analogue bugs:

- Primarily in I/O buffers, PLLs and thermal sensors

- Silicon does not operate in accordance with predicted (simulated) circuit behaviour